Bangladesh University of Engineering and Technology

Department of Computer Science and Engineering

CSE 200: Technical Writing and Presentation

Diffusion Models

An Overview of Progressive Denoising for Image Generation

Subsection: B2

Group: 08

Prepared by: Anindya Biswas ..... Student ID: 2205107
Tasnim Tamal ..... Student ID: 2205110
Din Mohammad ..... Student ID: 2205113

Date: April 10, 2026

Contents

Abstract **1**

1 Introduction **1**

2 The Diffusion Process **2**

2.1 Forward Diffusion Process 2

2.2 Reverse Diffusion Process 3

3 Model Architecture and Training **3**

3.1 Noise-Prediction Objective 4

3.2 Training Workflow 4

3.3 Sampling and Guidance in Practice 4

4 Why Diffusion Models Perform Well **5**

5 How Performance is Evaluated **5**

6 Important Practical Variants **6**

6.1 Noise Schedule and Sampling Intuition 6

6.2 Latent Diffusion 7

7 Applications **7**

8 Limitations and Responsible Use **8**

9 Conclusion **8**

List of Figures

1 Real and generated image comparison 1

2 Diffusion model overview 2

3 Progressive noising example 3

4 Simplified diffusion training diagram 3

5 Conceptual signal-noise balance over time 6

6 Latent diffusion pipeline 7

7 Representative diffusion-based applications 8

List of Tables

1 Notation summary for diffusion models 5
2 Comparison of common generative model families 5
3 Selected practical variants of diffusion models 7

Abstract

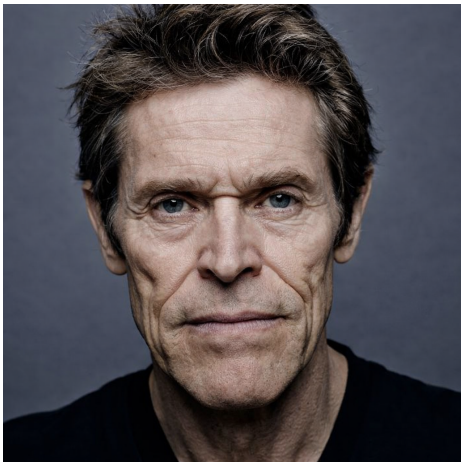
Diffusion models have become an important class of generative models because they can produce high-quality images without relying on adversarial training. Their basic idea is to gradually corrupt data with Gaussian noise and then learn how to reverse that process. This report explains the forward and reverse diffusion processes, the role of the U-Net architecture, and the standard noise-prediction objective used in Denoising Diffusion Probabilistic Models (DDPMs) [1, 2]. It also briefly discusses faster sampling methods, latent diffusion, common evaluation metrics, and several well-known applications [3, 4]. Finally, the report points out practical limitations such as slow sampling and high computational cost, along with broader concerns related to responsible use.

1 Introduction

Generative modeling is concerned with learning the underlying distribution of data so that new samples can be produced from it. For images, this is difficult because realistic output must preserve both fine texture and overall structure. Diffusion models are especially interesting in this setting because the core idea is simple, yet the results are strong in practice [1, 3]. This line of work is also closely related to score-based generative modeling, where noisy samples are used to learn an iterative refinement process [5].

Figure 1 shows why this topic matters. The generated face on the right looks plausible even though it does not belong to a real person. A useful generative model must therefore do more than memorize training examples; it must learn enough about the data distribution to create new samples that still look realistic.

(a) A real face sample.



(b) A model-generated face sample.

Figure 1: Example of a real image and a model-generated image. Good generative models aim to produce new samples that remain visually consistent with real data.

What makes diffusion models intuitive is that they do not try to generate the final image in a single step. During training, a real image is gradually corrupted with Gaussian noise, and the model learns to predict the noise added at different stages. During sampling,

the process is reversed: the model starts from random noise and gradually reconstructs structure. This stepwise formulation makes the learning problem easier to handle than direct one-shot image generation.

2 The Diffusion Process

The workflow of a diffusion model has two complementary parts: a *forward diffusion process* that destroys information and a *reverse diffusion process* that reconstructs structure. Figure 2 summarizes this idea in one diagram.

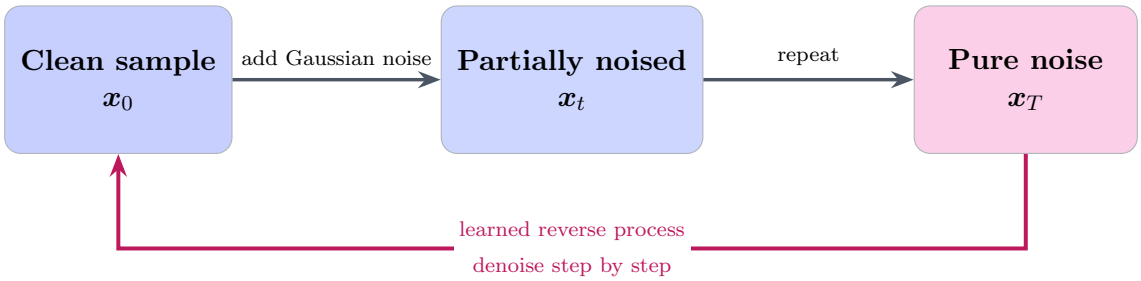


Figure 2: Conceptual overview of diffusion models. Training uses the controlled noising path, while sampling uses the learned reverse path.

2.1 Forward Diffusion Process

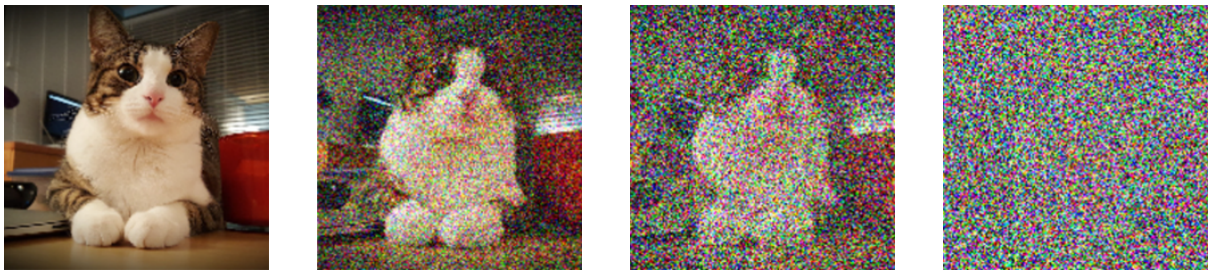
The forward process is a Markov chain that progressively perturbs a clean sample $\boldsymbol{x}_0 \in \mathbb{R}^d$ with Gaussian noise. At each timestep t, a small variance term β_t determines how much corruption is introduced:

$$q(\boldsymbol{x}_t \mid \boldsymbol{x}_{t-1}) = \mathcal{N}\left(\sqrt{1-\beta_t}\,\boldsymbol{x}_{t-1}, \beta_t \boldsymbol{I}\right). \tag{1}$$

If we define $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$, then a sample at any timestep can be written directly as

$$\boldsymbol{x}_t = \sqrt{\bar{\alpha}_t}\,\boldsymbol{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I}). \tag{2}$$

Equation 2 is convenient during training because it allows the model to sample a noisy state at any timestep directly, without simulating all previous steps.



(a) Clean (b) Low noise (c) Heavy noise (d) Pure noise

Figure 3: Visual interpretation of the forward diffusion process. A structured image is gradually transformed into an approximately Gaussian sample.

2.2 Reverse Diffusion Process

Once the data has been corrupted, the generative task becomes the reverse problem: infer how to move from $\boldsymbol{x}_t$ back to a less noisy sample $\boldsymbol{x}_{t-1}$. The model parameterized by θ learns a reverse distribution of the form

$$p_\theta(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t) = \mathcal{N}(\boldsymbol{\mu}_\theta(\boldsymbol{x}_t, t), \boldsymbol{\Sigma}_\theta(\boldsymbol{x}_t, t)) \,. \tag{3}$$

In practice, many implementations train a neural network to predict the added noise rather than predict clean pixels directly. This reparameterization simplifies learning because the network only needs to estimate the perturbation that separates $\boldsymbol{x}_t$ from the underlying clean sample [1, 2].

3 Model Architecture and Training

Besides the probabilistic formulation, the choice of network architecture also matters. Most image diffusion systems use a U-Net-style backbone with multi-scale feature extraction and skip connections [6]. The network takes two main inputs: a noisy image $\boldsymbol{x}_t$ and an embedding of the timestep t.

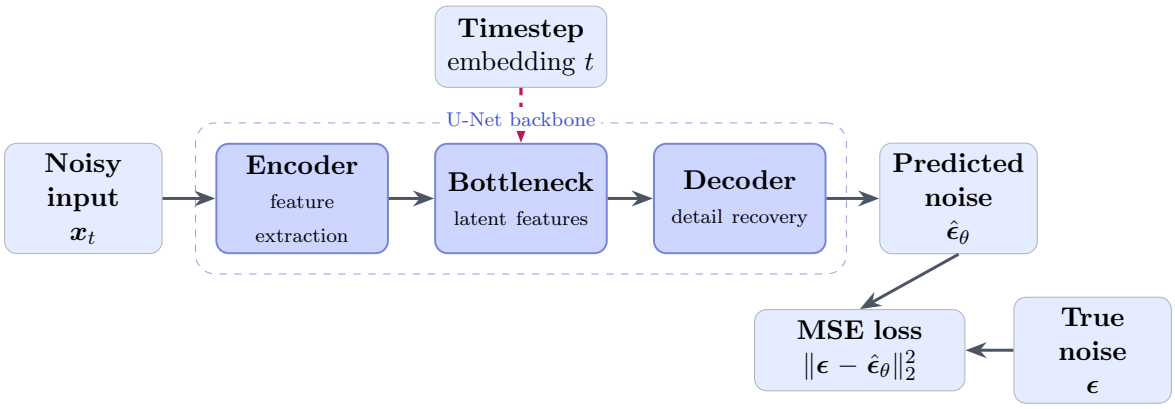


Figure 4: A simplified training view of an image diffusion model. The network receives a noisy image and the current timestep, predicts the noise component, and is trained with a mean-squared error objective.

3.1 Noise-Prediction Objective

The standard DDPM training objective is

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{\boldsymbol{x}_0,t,\boldsymbol{\epsilon}} \left[\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\boldsymbol{x}_t, t)\|_2^2 \right]. \tag{4}$$

This objective is convenient because the target noise $\boldsymbol{\epsilon}$ is known exactly; it is produced by the training procedure itself. That gives the model a clear supervision signal. In effect, the network is trained to solve many smaller denoising problems instead of one difficult image-generation problem.

3.2 Training Workflow

The complete training routine can be summarized as Algorithm 1. The procedure alternates between sampling a real image, corrupting it to a randomly chosen timestep, and updating the network so that its predicted noise matches the actual perturbation.

Algorithm 1 Training a diffusion model with noise prediction

Require: Training dataset $\mathcal{D}$, noise schedule $\{\beta_t\}_{t=1}^T$, neural network $\boldsymbol{\epsilon}_\theta$

```
while training has not converged do
    Sample a clean image x_0 ~ D
    Sample a timestep t uniformly from {1, ..., T}
    Sample Gaussian noise ε ~ N(0, I)
    Form a noisy input using Equation 2
    Predict ε̂ = ε_θ(x_t, t)
    Compute L_simple = ||ε − ε̂||_2^2
    Update θ using gradient descent
end while
```

3.3 Sampling and Guidance in Practice

At inference time, generation starts from $\boldsymbol{x}_T \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$ and repeatedly applies the learned reverse transitions until $\boldsymbol{x}_0$ is obtained. In practice, the main drawback is that this reverse process can be slow. One common way to reduce the number of steps is DDIM sampling [3]. Another practical idea is guidance, which pushes the model to follow a condition more strongly. Examples include classifier guidance and classifier-free guidance [7, 8]. These methods are especially important in modern text-to-image systems because they improve control over the final result.

Table 1 collects the most important symbols used in the mathematical discussion so that the notation remains easy to follow.

Symbol	Meaning
$\boldsymbol{x}_0$	A clean training sample drawn from the data distribution.
$\boldsymbol{x}_t$	The noisy version of the sample at diffusion step t.
β_t	Variance schedule controlling how much noise is added at step t.
α_t	Shorthand for $1 - \beta_t$.
$\bar{\alpha}_t$	Cumulative product $\prod_{s=1}^{t} \alpha_s$, measuring the remaining signal after t steps.
$\boldsymbol{\epsilon}$	Gaussian noise used to corrupt the sample.
$\boldsymbol{\epsilon}_\theta(\boldsymbol{x}_t, t)$	Neural network prediction of the added noise.

Table 1: Notation used in the forward process, reverse process, and training objective.

4 Why Diffusion Models Perform Well

There are several practical reasons diffusion models perform well. First, once Gaussian noise is sampled, the training target is known analytically, which avoids the adversarial instability often seen in GAN training. Second, iterative denoising lets coarse structure appear before fine details are refined. Third, conditioning inputs such as text, class labels, or masks can be integrated naturally into the denoising network, which makes the framework useful across many tasks.

These characteristics help explain why diffusion systems became competitive with, and in many settings preferable to, earlier generative approaches. Table 2 summarizes typical trade-offs.

Model family	Training stability	Sample quality	Sampling speed	Main comment
VAE	High	Moderate	Fast	Strong probabilistic interpretation but often blurrier outputs.
GAN	Moderate to low	High	Very fast	Can generate sharp images but training may be unstable and prone to mode collapse.
Diffusion model	High	Very high	Slow	Excellent fidelity and diversity, but generation requires many denoising steps.

Table 2: A high-level comparison of common generative model families. The table emphasizes typical tendencies rather than strict universal rules.

5 How Performance is Evaluated

When papers claim better image quality, they usually support those claims with both visual examples and numerical metrics. One of the most common metrics is the Fréchet

Inception Distance (FID), which compares feature statistics of generated and real images; lower values are generally better [9]. Many papers also report Inception Score (IS) and precision-recall style analyses to separate fidelity from diversity.

For conditional systems such as text-to-image models, image quality alone is not enough. The output should also follow the prompt while remaining realistic. In practice, this creates a trade-off between strict condition matching and natural-looking images, so most studies report multiple metrics rather than relying on only one.

6 Important Practical Variants

The basic DDPM formulation is enough to understand the core idea, but many real systems use modified samplers or compressed representations to make the method more practical. Two widely discussed examples are DDIM sampling [3] and latent diffusion [4]. These variants do not change the main idea of progressive denoising; instead, they make the framework more usable in real settings.

6.1 Noise Schedule and Sampling Intuition

Although the forward process is defined mathematically, it is helpful to think of diffusion in terms of how signal gradually gives way to noise. Early timesteps preserve most of the original structure, while later timesteps are dominated by noise. This transition is controlled by the schedule $\{\beta_t\}$ and the cumulative factor $\bar{\alpha}_t$.

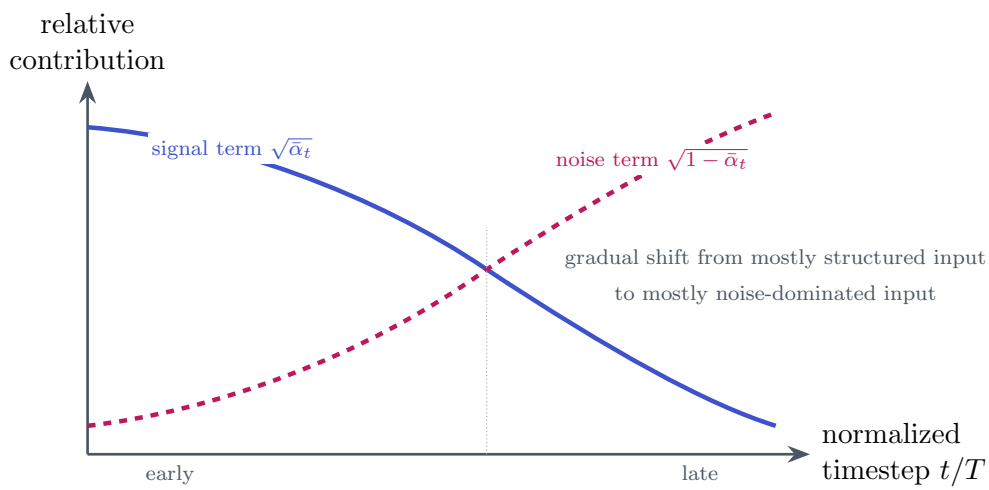


Figure 5: Conceptual signal-noise balance over diffusion timesteps. The plot is schematic and illustrates how the contribution of the original signal decreases while the contribution of noise increases.

Figure 5 is only a conceptual sketch, but it helps explain why sampling can be expensive: the model has to undo a long sequence of small changes. DDIM-style samplers address this issue by following a more direct denoising path, often producing useful samples with fewer steps than a standard DDPM sampler [3].

6.2 Latent Diffusion

Working directly in pixel space is expensive, especially for high-resolution images. Latent diffusion reduces this cost by encoding an image into a lower-dimensional latent representation, running diffusion in that space, and then decoding the result back into pixel space [4]. Because the denoising network operates on a compressed representation, both memory use and sampling cost can be reduced substantially.

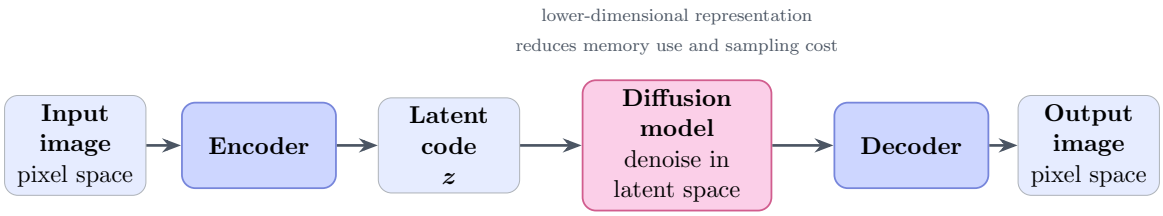


Figure 6: Simplified view of latent diffusion. Instead of applying diffusion directly in pixel space, the model denoises a compressed latent representation and then decodes the result back into an image.

This is one reason modern text-to-image systems can work at fairly high resolutions. The denoising process is still learned in the same way, but the computation is carried out in a more compact space. Table 3 summarizes the practical motivation behind several common variants.

Variant	Main idea	Practical benefit	Typical trade-off
DDPM	Learn the reverse diffusion chain in pixel space.	Strong baseline formulation and high sample quality.	Often requires many denoising steps.
DDIM	Use an implicit sampling path that can skip some steps.	Faster generation with fewer sampling steps.	Speed and quality depend on the chosen schedule.
Latent diffusion	Apply diffusion to a compressed latent representation.	Lower memory cost and more practical high-resolution synthesis.	Requires an additional encoder-decoder stage.

Table 3: Selected practical variants built on the basic diffusion framework.

7 Applications

Diffusion models are no longer only a research topic. They now appear both in popular creative tools and in more specialized technical workflows. In everyday applications, they are used for text-to-image synthesis, image editing, artistic rendering, and increasingly

for video generation. Figure 7 shows a few familiar systems associated with this broader family of methods.

(a) DALL-E. (b) Midjourney. (c) Sora.

Figure 7: Examples of widely recognized systems associated with diffusion-based generation. The figure is illustrative rather than exhaustive.

Beyond creative media, diffusion models are also becoming relevant in technical domains. In medical imaging, researchers use them for denoising, reconstruction, and data augmentation. In scientific design, related models are being explored for molecule generation and inverse problems. Latent diffusion models further reduce the cost of high-resolution synthesis by performing denoising in a compressed latent space rather than directly in pixel space [4].

8 Limitations and Responsible Use

Despite their strengths, diffusion models still have several important limitations. The first is **slow sampling**: high-quality generation may require dozens to hundreds of denoising steps, which increases latency. The second is **high computational cost**: training large systems demands substantial GPU memory, long training times, and significant electrical power. The third is **system complexity**: practical deployments often rely on careful prompt conditioning, safety filters, and post-processing pipelines.

There are also broader ethical concerns. Diffusion models can reproduce harmful biases present in their training data, imitate artistic styles without consent, or be used to create misleading synthetic media. For that reason, technical progress should be considered alongside dataset transparency, usage policies, and clear attribution practices.

9 Conclusion

Overall, diffusion models show how a difficult generation problem can be broken into a sequence of smaller denoising steps. That is what makes the framework both interesting to study and useful in practice. It brings together probability, optimization, neural networks, and real image-generation systems in a way that is easier to follow than many earlier generative approaches.

At the same time, the method is not free of cost: sampling is slow, training is expensive, and responsible use matters. Even with these limitations, diffusion models remain one of the most influential ideas in modern generative AI.

References

[1] J. Ho, A. N. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 6840–6851, 2020.

[2] A. Q. Nichol and P. Dhariwal, "Improved denoising diffusion probabilistic models," in *Proceedings of the 38th International Conference on Machine Learning*, vol. 139 of *Proceedings of Machine Learning Research*, pp. 8162–8171, PMLR, 2021.

[3] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," in *International Conference on Learning Representations*, 2021. OpenReview: St1giarCHLP.

[4] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 10684–10695, 2022.

[5] Y. Song and S. Ermon, "Generative modeling by estimating gradients of the data distribution," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[6] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention*, vol. 9351 of *Lecture Notes in Computer Science*, pp. 234–241, Springer, 2015.

[7] P. Dhariwal and A. Q. Nichol, "Diffusion models beat gans on image synthesis," in *Advances in Neural Information Processing Systems*, vol. 34, pp. 8780–8794, 2021.

[8] J. Ho and T. Salimans, "Classifier-free diffusion guidance," *arXiv*, vol. abs/2207.12598, 2022. Short version appeared in the NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications.

[9] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "Gans trained by a two time-scale update rule converge to a local nash equilibrium," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.